

Livello del Sistema Operativo

SINCRONIZZAZIONE DEI PROCESSI

Interazione tra processi

Processi concorrenti possono interagire

- Interazione → comportamento non riproducibile

Interazione tra thread o tra processi

Tipologie di interazione

- Cooperazione
- Competizione
- Interferenza

Cooperazione

Scambio di informazioni tra processi

- Es. scambio di dati
 - sincronizzazione nel modello produttore/consumatore

Sincronizzazione necessaria per ordinare l'esecuzione dei processi

- Es. il processo produttore non può produrre dati prima che il processo consumatore non abbia consumato i precedenti

Sincronizzazione esplicita perché l'interazione fa parte della logica dei processi

3

Competizione

Più processi richiedono l'uso di una risorsa comune che non può essere condivisa

- Es. risorse hardware (stampante)

Sincronizzazione necessaria per l'accesso mutuamente esclusivo delle risorse

- Non è previsto necessariamente un vincolo di precedenza

Sincronizzazione implicita perché l'interazione non fa parte della logica dei processi

- L'interazione si potrebbe eliminare disponendo di risorse aggiuntive

4

Interferenza



Interazione indesiderata tra processi causata da una cattiva sincronizzazione



Malfunzionamenti dipendenti dai rapporti di velocità tra i processi

Gli effetti erranei possono o meno verificarsi a seconda delle differenti condizioni di velocità di esecuzione dei processi (errori dipendenti dal tempo)



Obiettivo della sincronizzazione è di eliminare problemi di interferenza

5

Modelli di interazione

AMBIENTE GLOBALE

- Processi interagenti in una memoria comune
- I processi competono o cooperano per accedere alle risorse comuni
 - ogni risorsa descritta in memoria da strutture dati
 - es.: buffer o stampante (rappresentata da un descrittore)
- Sincronizzazione mediante **semafori**

AMBIENTE LOCALE

- Processi interagenti senza memoria comune (o in porzioni separate di memoria)
- Ogni processo gestisce risorse cui altri non possono accedere
 - Processi che necessitano di risorse inviano richieste
 - Processi che gestiscono risorse rispondono alle richieste
- Sincronizzazione mediante scambio di **messaggi**

6

Problema della mutua esclusione

Mutua Esclusione (ME): vincolo richiesto ai processi in competizione per l'uso di una risorsa non condivisibile

Il vincolo della ME richiede che le operazioni con cui i processi accedono alle risorse comuni non si sovrappongano nel tempo

La sequenza di operazioni di un processo soggette al vincolo della mutua esclusione si chiama sezione critica

7

Esempio di sezioni critiche

- Variabili condivise:

```
int stack[100];  
int top = 1;
```

- Estrazione:

```
int pop (void) {  
    int temp;  
    temp = stack[top];  
    top--;  
    return temp;  
}
```

- Inserimento:

```
void push (int y) {  
    top++;  
    stack[top]=y;  
}
```

- Esempio di interferenza fra due processi P1, P2:

```
T0: top++;           (P1)  
T1: temp=stack[top]; (P2)  
T2: top--;          (P2)  
T3: stack[top]=y;   (P1)
```

8

Soluzioni al problema della ME

DISABILITAZIONE DELLE INTERRUZIONI

- La disabilitazione avviene durante l'esecuzione delle sezioni critiche
 - Soluzione **parziale** (non applicabile in multiprocessori)
 - Soluzione **limitativa** (inibisce l'esecuzione di operazioni prioritarie e indipendenti dalla sezione critica)

ADOZIONE DI UN PROTOCOLLO DI SINCRONIZZAZIONE

- Un processo deve essere autorizzato prima di eseguire la sezione critica per una risorsa R
 - Le istruzioni che compongono la richiesta di autorizzazione prendono il nome di **prologo**
- Il SO non autorizza altri processi a entrare nelle loro sezioni critiche associate a R
 - Le istruzioni eseguite dal processo per dichiarare libera la sezione critica prendono il nome di **epilogo**

9

Ambiente globale - Uso di variabili di stato

occupato = 1 → risorsa occupata

occupato = 0 → risorsa libera

P₁

```
prologo: while (occupato==1);  
           occupato = 1;  
           <sezione critica A>;  
epilogo: occupato = 0;
```

P₂

```
prologo: while (occupato==1);  
           occupato = 1;  
           <sezione critica B>;  
epilogo: occupato = 0;
```

10

Problema con variabili di stato

- Esempio di esecuzione:
 - T_0 : P1 esegue while e trova occupato=0 (risorsa libera)
 - T_1 : P2 esegue while e trova occupato=0 (risorsa libera)
 - T_2 : P1 pone occupato=1 ed entra in A
 - T_3 : P2 pone occupato=1 ed entra in B
- Entrambi i processi sono **contemporaneamente** nella loro sezione critica

11

Test and Set Lock (TSL)

- **TSL Reg, x**
 - Il valore contenuto in x viene copiato nel registro R del processore e viene scritto in x un valore diverso da zero
 - Le operazioni di lettura e scrittura sono eseguite in modo indivisibile (hardware)
- Lettura e modifica di una parola in **un solo ciclo** di memoria
 - P1 può leggere e modificare la variabile occupato senza interferenze da parte di P2

12

Primitive lock e unlock

- La mutua esclusione nella esecuzione delle due sezioni critiche si ottiene introducendo le primitive **lock(x)** e **unlock(x)**

lock (x) :

```
TSL registro, x    (copia x nel registro e pone x=1)
CMP registro, 0    (il valore di x era 0 ? )
JNE lock          (se no, ricomincia il ciclo)
RET               (ritorna al chiamante)
```

unlock (x) :

```
MOV x, 0           (inserisce 0 in x)
RET               (ritorna al chiamante)
```

13

Soluzione al problema della ME con lock e unlock

P₁

prologo:

```
lock (x) ;
<sez. critica A>;
```

epilogo:

```
unlock (x) ;
```

P₂

prologo:

```
lock (x) ;
<sez. critica B>;
```

epilogo:

```
unlock (x) ;
```

- Svantaggio: **attesa attiva**
 - Generata dai cicli while nel prologo

14

Semafori

- Strutture dati messe a disposizione dal SO per la sincronizzazione
- Ogni **semaforo s** è costituito da
 - Una variabile intera non negativa (s.value)
 - Una coda di processi bloccati (s.queue)
- Ai semafori sono applicabili solo due primitive (indivisibili): **wait(s)** e **signal(s)**
- Garantiscono l'attesa passiva
 - Utilizzo più efficiente della CPU

15

Primitive wait e signal

```
void wait (semaphore s)

{ if (s.value==0)
    <il processo viene
    bloccato e il suo
    descrittore inserito in
    s.queue>;
  else
    s.value = s.value-1;
}
```

```
void signal (semaphore s)

{ if (<esiste almeno un
processo in s.queue>)
    <il descrittore del
    primo di questi viene
    estratto da s.queue e
    il suo stato modificato
    in pronto>;
  else
    s.value = s.value + 1;
}
```

16

Soluzione al problema della mutua esclusione con il semaforo

- Semaforo **mutex** associato alla risorsa condivisa
- Il semaforo è **binario** con valore iniziale: **mutex = 1**

```
prologo:      wait(mutex) ;  
              <sez. critica>;  
epilogo:     signal(mutex) ;
```

17

Utilizzo di un semaforo – Esempio

- Tre processi: P1, P2, P3 accedono a una stessa risorsa
 - Si indicano con SC_1, SC_2, SC_3 le loro sezioni critiche

```
P1  
prologo: wait(mutex) ;  
          <SC_1>;  
epilogo: signal(mutex) ;  
P2  
prologo: wait(mutex) ;  
          <SC_2>;  
epilogo: signal(mutex) ;  
P3  
prologo: wait(mutex) ;  
          <SC_3>;  
epilogo: signal(mutex) ;
```

```
Px  
prologo: wait(mutex) ;  
          <SC_x>;  
epilogo: signal(mutex) ;
```

18

Utilizzo di un semaforo – Esempio

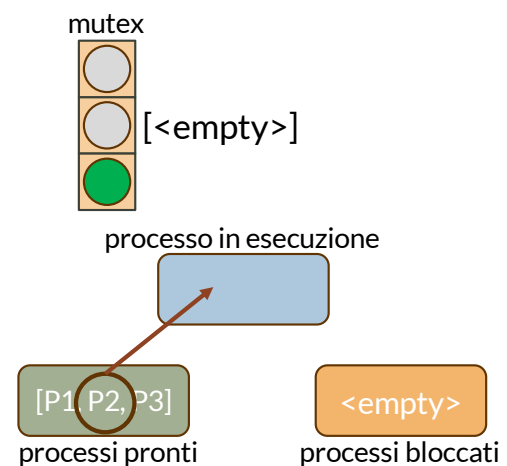
- Tre processi: P1, P2, P3 accedono a una stessa risorsa
 - Si indicano con SC_1, SC_2, SC_3 le loro sezioni critiche
- Valori iniziali di **mutex**:
 - **mutex.value** = 1
 - **mutex.queue** = <empty>
- Il primo processo ad accedere alla sezione critica è P2
 - P2 -> **wait(mutex)**
 - **mutex.value** = 0
 - P2 entra in SC_2

```
Px
prologo:  wait(mutex);
          <SC_x>;
epilogo:  signal(mutex);
```

19

Utilizzo di un semaforo – Esempio

- Possibile sequenza:
 - P2 → **wait(mutex)**
 - **mutex.value** = 1 → **mutex.value** = 0
 - P2 entra in SC_2
 - P2 passa allo stato pronto (revoca della CPU)
 - P1 passa allo stato esecuzione
 - P1 → **wait(mutex)**
 - **mutex.value** = 0
 - P1 passa allo stato bloccato
 - **mutex.queue** = [P1]
 - P3 passa allo stato esecuzione
 - P3 → **wait(mutex)**
 - **mutex.value** = 0
 - P3 passa allo stato bloccato
 - **mutex.queue** = [P1, P3]
 - P2 passa allo stato esecuzione e completa SC_2
 - P2 → **signal(mutex)**
 - P1 passa allo stato esecuzione e completa SC_1
 - P1 → **signal(mutex)**
 - P3 passa allo stato esecuzione e completa SC_3
 - P3 → **signal(mutex)**
 - **mutex.value** = 0 → **mutex.value** = 1



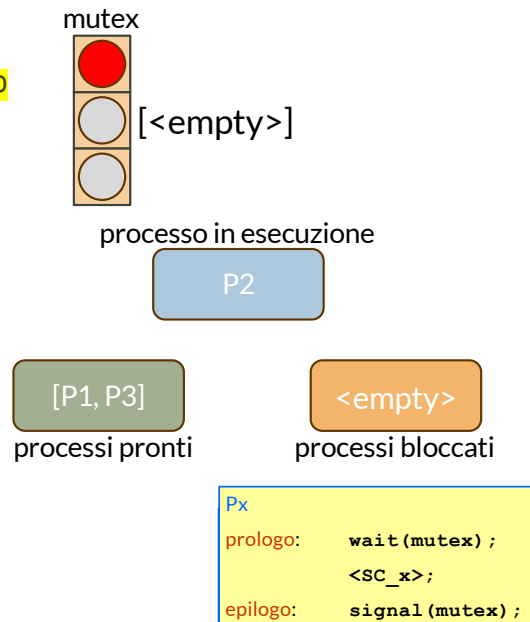
```
Px
prologo:  wait(mutex);
          <SC_x>;
epilogo:  signal(mutex);
```

20

Utilizzo di un semaforo – Esempio

- Possibile sequenza:

- **P2 → wait(mutex)**
- **mutex.value = 1 → mutex.value = 0**
- P2 entra in SC_2
- P2 passa allo stato pronto (revoca della CPU)
- P1 passa allo stato esecuzione
- P1 → wait(mutex)
- mutex.value = 0
- P1 passa allo stato bloccato
- mutex.queue = {P1}
- P3 passa allo stato esecuzione
- P3 → wait(mutex)
- mutex.value = 0
- P3 passa allo stato bloccato
- mutex.queue = {P1, P3}
- P2 passa allo stato esecuzione e completa SC_2
- P2 → signal(mutex)
- P1 passa allo stato esecuzione e completa SC_1
- P1 → signal(mutex)
- P3 passa allo stato esecuzione e completa SC_3
- P3 → signal(mutex)
- mutex.value = 0 → mutex.value = 1

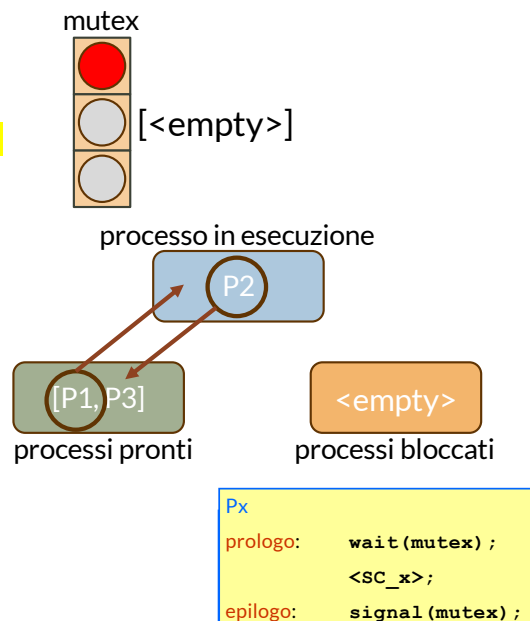


21

Utilizzo di un semaforo – Esempio

- Possibile sequenza:

- P2 → wait(mutex)
- mutex.value = 1 → mutex.value = 0
- **P2 entra in SC_2**
- **P2 passa allo stato pronto (revoca della CPU)**
- **P1 passa allo stato esecuzione**
- P1 → wait(mutex)
- mutex.value = 0
- P1 passa allo stato bloccato
- mutex.queue = {P1}
- P3 passa allo stato esecuzione
- P3 → wait(mutex)
- mutex.value = 0
- P3 passa allo stato bloccato
- mutex.queue = {P1, P3}
- P2 passa allo stato esecuzione e completa SC_2
- P2 → signal(mutex)
- P1 passa allo stato esecuzione e completa SC_1
- P1 → signal(mutex)
- P3 passa allo stato esecuzione e completa SC_3
- P3 → signal(mutex)
- mutex.value = 0 → mutex.value = 1

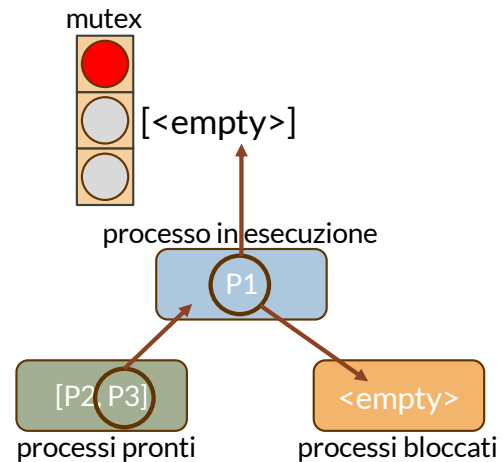


22

Utilizzo di un semaforo – Esempio

- Possibile sequenza:

- P2 → wait(mutex)
- mutex.value = 1 → mutex.value = 0
- P2 entra in SC_2
- P2 passa allo stato pronto (revoca della CPU)
- P1 passa allo stato esecuzione
- **P1 → wait(mutex)**
- **mutex.value = 0**
- **P1 passa allo stato bloccato**
- **mutex.queue = [P1]**
- **P3 passa allo stato esecuzione**
- P3 → wait(mutex)
- mutex.value = 0
- P3 passa allo stato bloccato
- mutex.queue = [P1, P3]
- P2 passa allo stato esecuzione e completa SC_2
- P2 → signal(mutex)
- P1 passa allo stato esecuzione e completa SC_1
- P1 → signal(mutex)
- P3 passa allo stato esecuzione e completa SC_3
- P3 → signal(mutex)
- mutex.value = 0 → mutex.value = 1



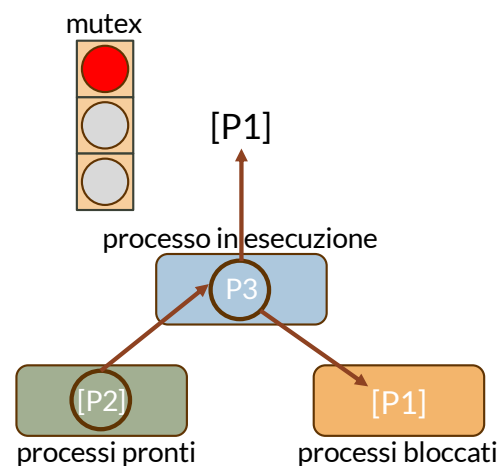
```
Px
prologo:  wait(mutex);
          <SC_x>;
epilogo:  signal(mutex);
```

23

Utilizzo di un semaforo – Esempio

- Possibile sequenza:

- P2 → wait(mutex)
- mutex.value = 1 → mutex.value = 0
- P2 entra in SC_2
- P2 passa allo stato pronto (revoca della CPU)
- P1 passa allo stato esecuzione
- P1 → wait(mutex)
- mutex.value = 0
- P1 passa allo stato bloccato
- mutex.queue = [P1]
- P3 passa allo stato esecuzione
- **P3 → wait(mutex)**
- **mutex.value = 0**
- **P3 passa allo stato bloccato**
- **mutex.queue = [P1, P3]**
- **P2 passa allo stato esec. e completa SC_2**
- P2 → signal(mutex)
- P1 passa allo stato esecuzione e completa SC_1
- P1 → signal(mutex)
- P3 passa allo stato esecuzione e completa SC_3
- P3 → signal(mutex)
- mutex.value = 0 → mutex.value = 1



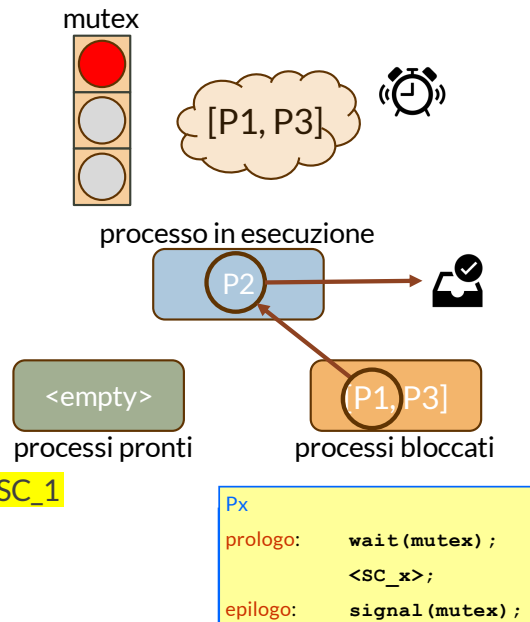
```
Px
prologo:  wait(mutex);
          <SC_x>;
epilogo:  signal(mutex);
```

24

Utilizzo di un semaforo – Esempio

- Possibile sequenza:

- P2 → wait(mutex)
 - mutex.value = 1 → mutex.value = 0
- P2 entra in SC_2
- P2 passa allo stato pronto (revoca della CPU)
- P1 passa allo stato esecuzione
- P1 → wait(mutex)
 - mutex.value = 0
- P1 passa allo stato bloccato
- mutex.queue = [P1]
- P3 passa allo stato esecuzione
- P3 → wait(mutex)
 - mutex.value = 0
- P3 passa allo stato bloccato
- mutex.queue = [P1, P3]
- P2 passa allo stato esecuzione e completa SC_2
- **P2 → signal(mutex)**
- **P1 passa allo stato esecuzione e completa SC_1**
- P1 → signal(mutex)
- P3 passa allo stato esecuzione e completa SC_3
- P3 → signal(mutex)
- mutex.value = 0 → mutex.value = 1

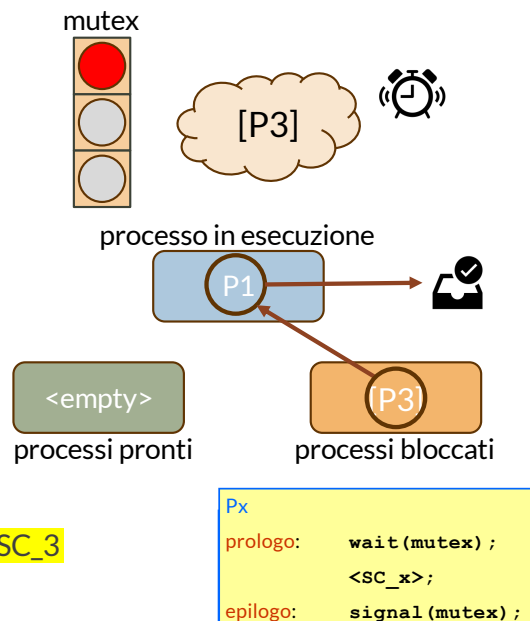


25

Utilizzo di un semaforo – Esempio

- Possibile sequenza:

- P2 → wait(mutex)
 - mutex.value = 1 → mutex.value = 0
- P2 entra in SC_2
- P2 passa allo stato pronto (revoca della CPU)
- P1 passa allo stato esecuzione
- P1 → wait(mutex)
 - mutex.value = 0
- P1 passa allo stato bloccato
- mutex.queue = [P1]
- P3 passa allo stato esecuzione
- P3 → wait(mutex)
 - mutex.value = 0
- P3 passa allo stato bloccato
- mutex.queue = [P1, P3]
- P2 passa allo stato esecuzione e completa SC_2
- P2 → signal(mutex)
- P1 passa allo stato esecuzione e completa SC_1
- **P1 → signal(mutex)**
- **P3 passa allo stato esecuzione e completa SC_3**
- P3 → signal(mutex)
- mutex.value = 0 → mutex.value = 1

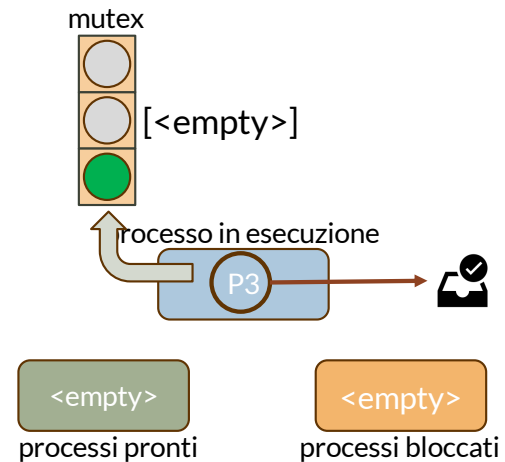


26

Utilizzo di un semaforo – Esempio

- Possibile sequenza:

- P2 → wait(mutex)
 - mutex.value = 1 → mutex.value = 0
- P2 entra in SC_2
- P2 passa allo stato pronto (revoca della CPU)
- P1 passa allo stato esecuzione
- P1 → wait(mutex)
 - mutex.value = 0
- P1 passa allo stato bloccato
 - mutex.queue = [P1]
- P3 passa allo stato esecuzione
- P3 → wait(mutex)
 - mutex.value = 0
- P3 passa allo stato bloccato
 - mutex.queue = [P1, P3]
- P2 passa allo stato esecuzione e completa SC_2
- P2 → signal(mutex)
- P1 passa allo stato esecuzione e completa SC_1
- P1 → signal(mutex)
- P3 passa allo stato esecuzione e completa SC_3
- **P3 → signal(mutex)**
- **mutex.value = 0 → mutex.value = 1**



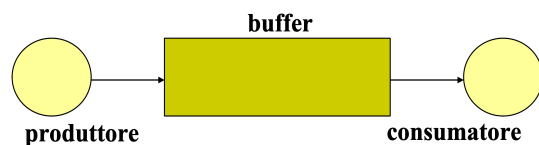
Px
 prologo: wait(mutex) ;
 <SC_x>;
 epilogo: signal(mutex) ;

27

Il problema della comunicazione

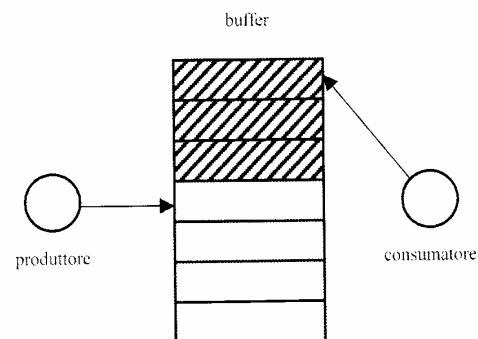
- Modello **produttore/consumatore**

- Produttore: processo che genera ciclicamente dati depositati in un buffer
- Consumatore: processo che preleva ciclicamente dati da un buffer



- Vincoli:

- Il consumatore non può prelevare dati dal buffer se questo è vuoto
- Il produttore non può depositare dati nel buffer se questo è pieno



28

Soluzione mediante semafori

- Due semafori:
 - **spazio_disponibile** - valore iniziale 1 (oppure N, se il buffer ha N locazioni)
 - **messaggio_disponibile** - valore iniziale 0

```
produttore{  
  
    <produzione del nuovo mess.>;  
    wait(spazio_disponibile);  
    <deposito del mess. nel buffer>;  
    signal(messaggio_disponibile);  
}
```

```
consumatore{  
  
    wait(messaggio_disponibile);  
    <prelievo del mess. dal buffer>;  
    signal(spazio_disponibile);  
    <consumo mess.>;  
}
```

29

Ambiente locale - Comunicazione mediante messaggi

IPC (Inter-Process Communication)

- Meccanismo offerto dal kernel per attuare la comunicazione fra processi

Primitive send e receive

- send(destinazione/i, messaggio)
- receive(origine, messaggio)

Impiego

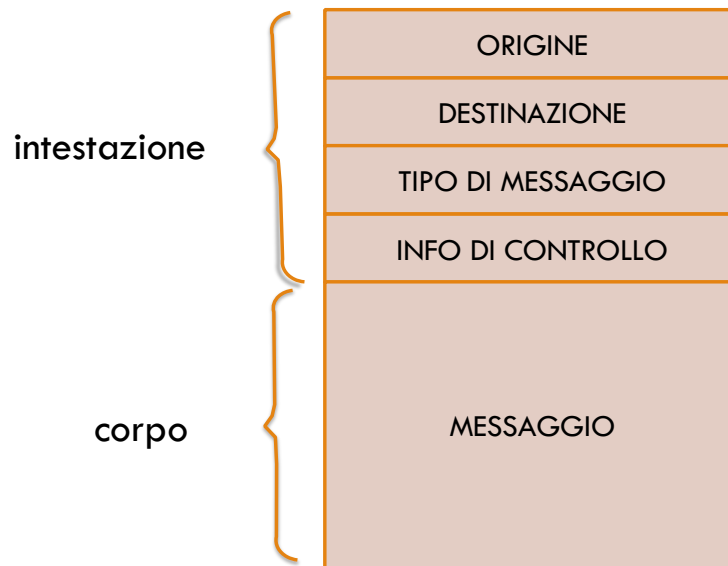
- Sistemi distribuiti/multicomputer
- Sistemi multiprocessore
- Sistemi monoprocessore

Canale di comunicazione

- Fisico (rete, bus, ...) nel caso di sistemi privi di memoria comune
- Logico (area di memoria) nel caso di sistemi con memoria comune

30

Messaggi



31

Comunicazione diretta

COMUNICAZIONE DIRETTA SIMMETRICA

- Si stabilisce un collegamento diretto fra coppie di processi P, Q
- I processi devono conoscere l'identificativo dell'altro processo coinvolto nella comunicazione
 - `send(P, message)`
 - `receive(Q, message)`

COMUNICAZIONE DIRETTA ASIMMETRICA

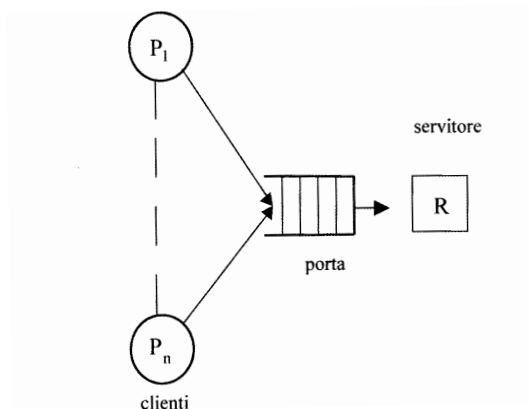
- Solo il mittente esplicita il nome del ricevente
- Il ricevente può ricevere messaggi da parte di qualunque processo
- Utilizzo di una variabile generica
 - `send(P, message)`
 - `receive(id, message)`

Svantaggio: una modifica del nome del processo implica la revisione di tutte le operazioni di comunicazione nelle quali il processo è coinvolto

32

Comunicazione indiretta

- La comunicazione tra due processi avviene tramite mailbox (porte):
 - `send(mailbox, message)`
 - `receive(mailbox, message)`
- Relazione tra mittenti e riceventi di tipo **multi-a-uno**



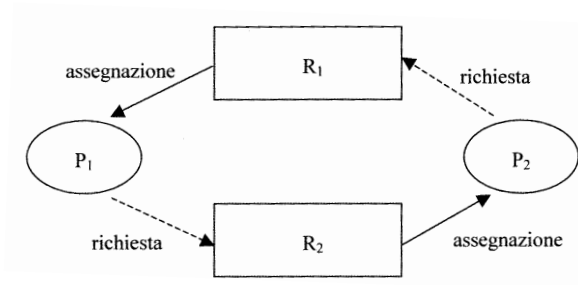
33

Blocco critico (stallo di processi)

- | | |
|------------------------------|------------------------------|
| • P1 | • P2 |
| ... | ... |
| <code>wait(mutex1);</code> | <code>wait(mutex2);</code> |
| ... uso di R1 | ... uso di R2 |
| <code>wait(mutex2);</code> | <code>wait(mutex1);</code> |
| ... uso di R2 | ... uso di R1 |
| <code>signal(mutex2);</code> | <code>signal(mutex1);</code> |
| ... rilascio di R2; | ... rilascio di R1; |
| <code>signal(mutex1);</code> | <code>signal(mutex2);</code> |
| ... rilascio di R1 | ... rilascio di R2 |
| ... | ... |

34

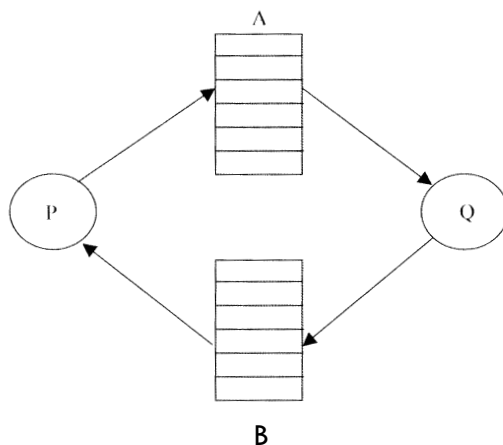
Stallo



- Creazione di una “catena circolare”
- La condizione di stallo dipende anche dalla **velocità relativa** di esecuzione dei processi
- Se P_1 avesse acquisito anche R_2 prima di P_2 lo stallo non sarebbe avvenuto

36

Stallo su risorse consumabili



- P è produttore per A e consumatore per B
- Q è produttore per B e consumatore per A
- Se i due buffer A e B sono pieni, P e Q sono bloccati in attesa dello svuotamento

37

Stallo: considerazioni

- Evento **raro**
 - Può occorrere dopo anni di funzionamento corretto
- Può provocare **gravi malfunzionamenti**
 - Tende a bloccare gradualmente tutti i processi
- La maggior parte dei SO **non offre** strumenti di prevenzione da situazioni di stallo
 - Prevenzione necessaria nei sistemi mission-critical

38

Condizioni di stallo

- Mutua esclusione
 - Le risorse possono essere utilizzate solo da un processo alla volta
- Possesso e attesa
 - I processi trattengono le risorse che già possiedono mentre chiedono risorse aggiuntive
- Mancanza di pre-rilascio
 - Le risorse già assegnate ai processi non possono essere a questi sottratte
- Attesa circolare

39

Metodi di trattamento degli stalli

- Prevenzione
 - Statica
 - Dinamica (algoritmo del banchiere)
- Rilevamento e ripristino da situazioni di stallo
- **Nessuno**
 - Adottato nei sistemi operativi per sistemi di fascia bassa

40

Prevenzione statica

- Assicurarsi durante la scrittura dei programmi che almeno una delle condizioni di stallo non sia verificata
- **Possesso e attesa**
 - Acquisizione contemporanea di tutte le risorse necessarie a un processo
 - Necessario conoscere tutte le risorse necessarie in anticipo
 - Lunghi tempi di attesa del processo per l'ottenimento totale
 - Bassa utilizzazione delle risorse
- **Mancanza di pre-rilascio**
 - Se un processo richiede una risorsa ma questa non gli viene assegnata, il processo rilascia tutte le risorse e si riavvia
 - Se un processo richiede una risorsa ma questa è impegnata da un secondo processo, il S.O. sottrae la risorsa a quest'ultimo e lo riavvia
- **Attesa circolare**
 - Risorse ordinate in forma gerarchica
 - Se un processo richiede una risorsa a livello d, può chiedere successivamente solo risorse di livello superiore

41

Prevenzione dinamica

- Si basa su algoritmi che verificano se il soddisfacimento di una particolare richiesta può provocare una situazione di stallo
- Algoritmo del banchiere
 - Verifica se, a fronte dell'eventuale soddisfacimento di una richiesta di assegnazione di una risorsa, esiste una sequenza di esecuzione per la quale tutti i processi possono completare l'esecuzione utilizzando le risorse libere e quelle che man mano vengono liberate

42

Algoritmo del banchiere

- Informazioni note:
 - num. tot. risorse: m
 - num. max di risorse richieste dai processi: $\max[P_i]$ ($i = 1 \dots n$)
- Informazioni ricavabili in ogni istante:
 - num. risorse già assegnate a un processo: $\text{assegnazione}[P_i]$
 - num. risorse ancora da chiedere: $\text{necessità}[P_i]$
 - num. risorse non utilizzate: $\text{disponibili} = m - \sum_i (\text{assegnazione}[P_i])$
- Esempio di configurazione iniziale (3 processi, 15 risorse):

- | | |
|----------------------------------|--|
| • $\text{assegnazione}[P_1] = 0$ | • $\text{necessità}[P_1] = \max[P_1] = 12$ |
| • $\text{assegnazione}[P_2] = 0$ | • $\text{necessità}[P_2] = \max[P_2] = 3$ |
| • $\text{assegnazione}[P_3] = 0$ | • $\text{necessità}[P_3] = \max[P_3] = 11$ |
| • $\text{disponibili} = m = 15$ | |

43

Algoritmo del banchiere

- Possibile configurazione (tempo t)
 - assegnazione[P_1] = 8
 - assegnazione[P_2] = 2
 - assegnazione[P_3] = 3
 - necessità[P_1] = 4
 - necessità[P_2] = 1
 - necessità[P_3] = 8
 - disponibili = $15 - (8 + 2 + 3) = 2$
- Lo stato è **sicuro**
 - (P_2, P_1, P_3) è una sequenza che assicura la terminazione di tutti i processi
- Cosa accadrebbe se a partire da questo stato fosse soddisfatta la richiesta di una risorsa da parte di P_3 ?

$\max[P_1] = 12$
$\max[P_2] = 3$
$\max[P_3] = 11$

44

Individuazione di blocchi critici

- Individuazione
 - Esecuzione periodica di algoritmi per la verifica della sussistenza di blocchi critici
 - Verifica simulando l'assegnazione delle risorse richieste
 - Verifica del grado di utilizzazione della CPU
- Ripristino
 - Aborto di tutti i processi in stallo
 - Aborto di un processo per volta
 - Sottrazione di risorse a processi
 - Uso di checkpoint per evitare il riavvio completo del processo ed eventuali perdite di dati

45